

GDA Doctrine Automation Workflow Outline

Last updated: 2026-05-08

Goal

Automate sprint doctrine capture and publication so merged work drafts itself automatically, and a gated "Finalize Sprint" action publishes canonical documentation only when platform checks are green.

Workflow 1: PR Merge -> Draft Doctrine

Trigger: GitHub PR merged to `main`

Steps

1. Receive GitHub webhook.
2. Parse PR metadata:
 - PR number
 - title
 - body
 - labels
 - merged commit SHA
 - changed files
3. Determine:
 - `sprint_id` (example: S-206)
 - component (example: Approvals Queue, QA Center)
 - doctrine impact
 - safety lane impact if any
4. Generate normalized doctrine sections:
 - current live state
 - planned state
 - recommended next state
 - doctrine / decision text
5. Upsert one or more rows into `doctrine_drafts`.
6. Rebuild the sprint draft markdown from Postgres rows.
7. Write or update a GitHub draft file such as `docs/sprints/S-206-draft.md`.

8. Record run result in `doctrine_publish_runs` with `trigger_type = 'pr-merge'`.

Result

- Postgres holds the canonical draft state.
- GitHub shows the current visible draft.
- No human editing required.

Workflow 2: Finalize Sprint

Trigger: manual n8n run with `sprintId`

Required gates

Before publishing canon docs, verify all required checks are green:

1. GitHub/CI build status is green.
2. Required React smoke checks are green.
3. QA Center read-only checks are green.
4. QA Center dry-run checks are green for the required workflows.
5. Master Build guard checks pass:
 - frozen workflows remain untouched
 - no critical write-capable workflow is missing a lane

If any gate fails:

- stop publish
- write a blocked run to `doctrine_publish_runs`
- emit a plain-English reason

Publish steps

1. Read all `doctrine_drafts` rows for the sprint.
2. Build final markdown for `docs/sprints/S-XXX.md`.
3. Build Book of Truths update block(s).
4. Commit updates to GitHub using server-side credentials.
5. Mark relevant rows `finalized` and store the resulting commit SHA.
6. Record success in `doctrine_publish_runs`.
7. Optionally trigger gist mirror update after GitHub succeeds.

Suggested n8n nodes

PR Merge -> Draft Doctrine

- GitHub Trigger
- Function: parse sprint/component metadata
- Function: classify doctrine impact
- Postgres: upsert doctrine_drafts
- Postgres: insert doctrine_publish_runs
- Postgres: select sprint rows
- Function: render markdown
- GitHub API HTTP Request: create/update S-XXX-draft.md
- Postgres: update publish run status

Finalize Sprint

- Manual Trigger / Webhook
- Postgres: insert running doctrine_publish_runs row
- HTTP Request: CI / build status
- HTTP Request: QA Center health
- Postgres or file read: Master Build checks
- IF node: all checks passed?
- Postgres: select doctrine_drafts rows for sprint
- Function: render final sprint markdown
- Function: render Book of Truths patch
- GitHub API HTTP Request: update final docs
- Postgres: mark rows finalized
- Postgres: update publish run status
- Optional: call gist mirror workflow
- Notify node

What this gives GDA

- Documentation updates happen as part of the operating system, not as manual cleanup.
- Postgres stores the real draft state.
- GitHub remains the canonical home for long-lived docs.
- Finalized docs are gated by actual platform health, not optimism.

